

Chapter 25

Multi-Agent Systems

25.1 Motivation: Why Multiple Agents?

The history of artificial intelligence is, in many ways, a history of scale. Early AI systems were monolithic: a single program, a single knowledge base, a single inference engine. As problems grew more complex, researchers discovered that no single agent—however capable—could efficiently handle every aspect of a rich, open-ended task. This insight, long established in distributed AI and multi-agent systems (MAS) research [375, 381], has found renewed urgency in the era of large language models.

The Core Intuition

A single LLM, no matter how large, is a generalist. A team of specialized LLMs, each focused on a narrow sub-problem and communicating their results, can outperform the generalist on complex, multi-faceted tasks—just as a team of human specialists outperforms a single generalist on a complex engineering project.

Four fundamental motivations drive the shift from monolithic agents to *agent societies*:

Specialization. Different sub-tasks benefit from different capabilities, prompting strategies, and even different base models. A code-generation agent can be fine-tuned on programming corpora; a fact-checking agent can be grounded with retrieval tools; a creative-writing agent can be prompted for stylistic diversity. Forcing a single agent to excel at all of these simultaneously is both inefficient and often impossible.

Parallelism. Many real-world tasks decompose into independent sub-tasks that can be executed concurrently. A research pipeline that requires literature review, data analysis, and report writing can run all three in parallel, dramatically reducing wall-clock time. Sequential single-agent processing is a bottleneck that multi-agent parallelism eliminates.

Robustness. A single agent is a single point of failure. If it hallucinates, gets stuck in a loop, or produces a subtly wrong answer, there is no check. Multi-agent systems introduce redundancy: a second agent can verify, critique, or independently re-derive results. Adversarial agents can probe for weaknesses before outputs are trusted.

Emergent Capabilities. Perhaps most intriguingly, agent collectives can exhibit capabilities that no individual agent possesses. Through debate, negotiation, and iterative refinement, multi-agent systems can arrive at solutions that transcend what any single agent could produce alone—a computational analog to the emergent intelligence of social organisms.

Historical Context

Multi-agent systems research dates to the 1980s, with foundational work on distributed problem solving [80], the Contract Net Protocol [334], and FIPA agent communication standards [93]. The shift to LLM-based agents reanimates these classical ideas with a new substrate: instead of

hand-coded agents with symbolic reasoning, we now have agents whose “cognition” emerges from learned neural representations. The core architectural patterns—hierarchies, markets, blackboards, message passing—remain remarkably relevant.

The transition from monolithic agents to agent societies mirrors a broader pattern in complex systems: as the problem space grows, distributed, modular architectures consistently outperform centralized, monolithic ones. The question is no longer *whether* to use multiple agents, but *how* to organize them.

25.2 Multi-Agent Architectures

The topology of a multi-agent system—how agents are connected and how authority flows among them—is the most consequential architectural decision. Four canonical patterns have emerged, each with distinct trade-offs.

25.2.1 Centralized (Supervisor/Manager) Architecture

In a centralized architecture, a single *orchestrator* agent (variously called supervisor, manager, or planner) holds global state, decomposes tasks, delegates sub-tasks to worker agents, and aggregates their results. The topology is a **hub-and-spoke**: all communication flows through the central node.

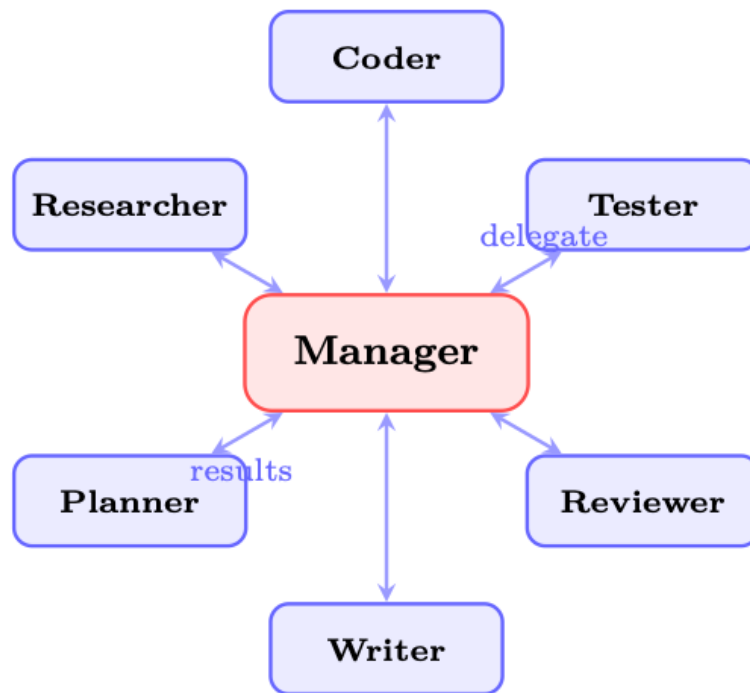


Figure 25.1: Centralized (Supervisor) architecture. The manager delegates tasks to specialized workers and aggregates their outputs. All communication flows through the central hub.

The manager’s responsibilities include:

- **Task routing:** deciding which worker is best suited for each sub-task
- **Context management:** providing each worker with the relevant subset of global context
- **Result aggregation:** synthesizing worker outputs into a coherent whole
- **Error handling:** detecting worker failures and re-routing or retrying

Supervisor Pattern in LangGraph

```

from langgraph.graph import StateGraph, START, END
from typing import TypedDict, Literal

class TeamState(TypedDict):
    task: str
    plan: str
    code: str
    tests: str
    review: str
    next_agent: str
    final_output: str

def supervisor_node(state: TeamState) -> TeamState:
    """Central orchestrator: decides which agent to invoke next."""
    messages = [
        {"role": "system", "content": SUPERVISOR_PROMPT},
        {"role": "user", "content": f"Task: {state['task']}\n"
                                     f"Plan: {state.get('plan', '')}\n"
                                     f"Code: {state.get('code', '')}\n"
                                     f"Tests: {state.get('tests', '')}\n"
                                     "Which agent should act next? "
                                     "Options: planner, coder, tester, "
                                     "reviewer, FINISH"}
    ]
    response = llm.invoke(messages)
    return {**state, "next_agent": response.content.strip()}

def route(state: TeamState) -> Literal["planner", "coder", "tester", "reviewer",
    __end__]:
    return state["next_agent"] if state["next_agent"] != "FINISH" else END

builder = StateGraph(TeamState)
builder.add_node("supervisor", supervisor_node)
builder.add_node("planner", planner_node)
builder.add_node("coder", coder_node)
builder.add_node("tester", tester_node)
builder.add_node("reviewer", reviewer_node)

builder.add_edge(START, "supervisor")
builder.add_conditional_edges("supervisor", route)
for agent in ["planner", "coder", "tester", "reviewer"]:
    builder.add_edge(agent, "supervisor") # always return to supervisor

graph = builder.compile()

```

Centralized Architecture Trade-offs

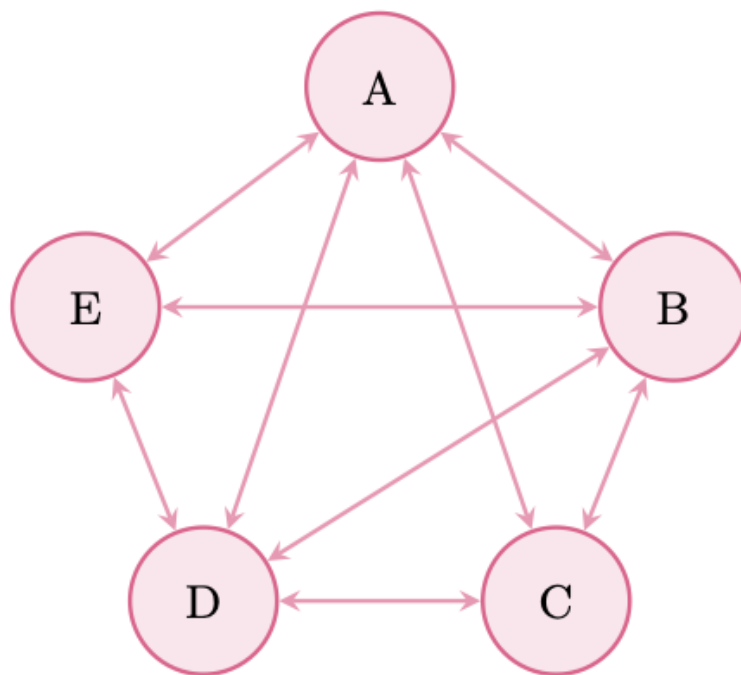
Pros: Simple control flow; clear accountability; easy to debug (all decisions in one place); straightforward to implement.

Cons: Single point of failure—if the manager hallucinates or gets confused, the entire system fails; the manager becomes a bottleneck under high load; the manager's context window must hold the global state, limiting scalability.

25.2.2 Decentralized (Peer-to-Peer) Architecture

In a decentralized architecture, agents interact directly with one another without a central coordinator. The topology is a **mesh**: any agent can communicate with any other. Coordination emerges from local interactions rather than global planning.

Emergent coordination in peer-to-peer systems arises through mechanisms such as:



Mesh (Peer-to-Peer) Topology

Figure 25.2: Decentralized (peer-to-peer) architecture. Agents communicate directly; coordination emerges from local interactions.

- **Negotiation:** agents bid for tasks or resources
- **Stigmergy:** agents modify shared state that others observe (see Section 25.3.6)
- **Gossip protocols:** agents propagate information through the network
- **Local consensus:** small groups of agents reach agreement without global coordination

Decentralized Architecture Trade-offs

Pros: Resilient to individual agent failures; scales naturally as agents are added; no bottleneck.

Cons: Hard to debug—emergent behavior is difficult to trace; potential for conflicts when agents have inconsistent views of state; coordination overhead grows as $O(n^2)$ with naive message passing; difficult to guarantee global consistency.

25.2.3 Hierarchical Architecture

Hierarchical architectures generalize the centralized pattern into a **tree structure** with multiple levels of management. A top-level orchestrator delegates to domain-specific sub-managers, who in turn delegate to specialized workers. This mirrors the organizational structure of large enterprises.

Key features of hierarchical systems:

- **Delegation chains:** authority and context flow down the tree; results flow up
- **Escalation paths:** workers can escalate unresolvable issues to their manager
- **Domain isolation:** sub-managers maintain domain-specific context, reducing the cognitive load on the top-level orchestrator
- **Scope limitation:** each agent only needs to know about its immediate superiors and subordinates

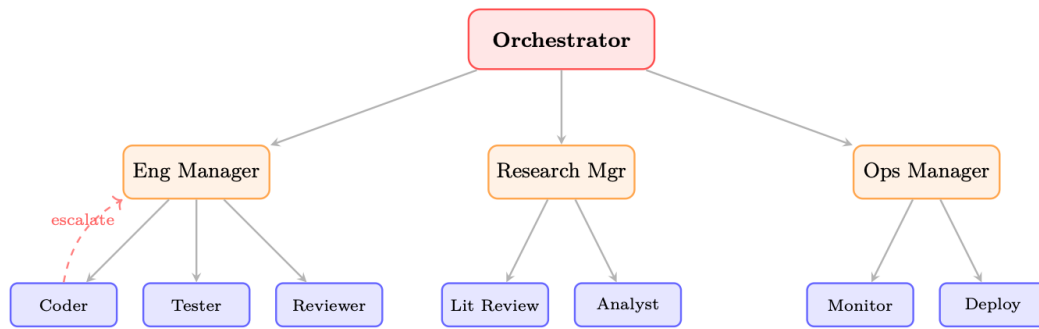


Figure 25.3: Hierarchical architecture. A top-level orchestrator delegates to domain sub-managers, who delegate to specialized workers. Dashed arrow shows an escalation path.

The enterprise analogy is apt: a CEO (top orchestrator) sets strategy; VPs (sub-managers) translate strategy into domain plans; individual contributors (workers) execute. The hierarchy enables scale while preserving accountability.

25.2.4 Swarm Architecture

Swarm architectures, inspired by biological systems (ant colonies, bird flocking), consist of many **loosely coupled agents** that follow simple local rules, producing complex global behavior without any central coordinator or global state.

OpenAI's **Swarm** framework [276] (now superseded by the OpenAI Agents SDK, but its conceptual primitives remain influential) operationalizes this with two primitives:

- **Routines:** sequences of instructions an agent follows to complete a sub-task
- **Handoffs:** an agent transferring control (and relevant context) to another agent

OpenAI Swarm: Routines and Handoffs

```

from swarm import Swarm, Agent

client = Swarm()

def transfer_to_billing():
    """Handoff: transfer control to the billing specialist."""
    return billing_agent

def transfer_to_technical():
    """Handoff: transfer control to the technical support agent."""
    return technical_agent

triage_agent = Agent(
    name="Triage Agent",
    instructions="""You are a customer service triage agent.
Determine the nature of the customer's issue:
- For billing questions, transfer to billing.
- For technical issues, transfer to technical support.
- For general questions, answer directly.""",
    functions=[transfer_to_billing, transfer_to_technical],
)

billing_agent = Agent(
    name="Billing Specialist",
    instructions="""You handle billing inquiries. "
    "Access account data and resolve payment issues.""",
    functions=[lookup_account, process_refund],
)

technical_agent = Agent(

```

```

name="Technical Support",
instructions="You resolve technical issues. "
            "Diagnose problems and provide step-by-step solutions.",
functions=[run_diagnostics, escalate_to_engineering],
)

# No global state --- each agent operates on its local context
response = client.run(
    agent=trriage_agent,
    messages=[{"role": "user", "content": "My invoice is wrong"}]
)

```

Swarm Properties

- **No global state:** each agent maintains only its local context window
- **Local decisions:** routing decisions are made by the current agent, not a central planner
- **Task completion through collective behavior:** complex tasks are completed through a chain of handoffs, each agent contributing its specialty
- **Lightweight:** no orchestration overhead; agents are stateless between handoffs

25.3 Coordination Mechanisms

How agents coordinate—how they share information, divide work, and resolve conflicts—is as important as the topology. Six canonical coordination mechanisms apply to LLM-based multi-agent systems.

25.3.1 Shared State (Global Blackboard)

The **blackboard architecture** [135] provides a shared data structure that all agents can read from and write to. In LLM systems, this is typically implemented as a shared dictionary, database, or structured document.

```

import threading
from dataclasses import dataclass, field
from typing import Any, Callable, Dict, List

@dataclass
class BlackboardEntry:
    value: Any
    author: str
    timestamp: float
    confidence: float = 1.0

class Blackboard:
    """Thread-safe shared state for multi-agent coordination."""

    def __init__(self):
        self._data: Dict[str, BlackboardEntry] = {}
        self._lock = threading.RLock()
        self._subscribers: Dict[str, List[Callable]] = {}

    def write(self, key: str, value: Any, author: str,
              confidence: float = 1.0) -> bool:
        """Write to blackboard; higher-confidence entries win conflicts."""
        with self._lock:
            existing = self._data.get(key)
            if existing and existing.confidence > confidence:
                return False # Conflict: existing entry wins
            import time

```

```

        self._data[key] = BlackboardEntry(
            value=value, author=author,
            timestamp=time.time(), confidence=confidence
        )
        self._notify(key, value)
        return True

    def read(self, key: str) -> Any:
        with self._lock:
            entry = self._data.get(key)
            return entry.value if entry else None

    def subscribe(self, key: str, callback: Callable):
        """Agents subscribe to changes on specific keys."""
        self._subscribers.setdefault(key, []).append(callback)

    def _notify(self, key: str, value: Any):
        for cb in self._subscribers.get(key, []):
            cb(key, value)

```

Listing 25.1: Shared blackboard with conflict resolution

25.3.2 Message Passing

Message passing is the most natural coordination mechanism for LLM agents: agents communicate by sending structured text messages to one another. Key design decisions include:

- **Message format:** structured (JSON schema) vs. natural language vs. hybrid
- **Routing:** direct (agent-to-agent) vs. broadcast vs. topic-based pub/sub
- **Conversation threads:** maintaining context across multi-turn exchanges
- **Acknowledgment:** whether senders require confirmation of receipt/processing

25.3.3 Planning and Decomposition

A manager agent decomposes a high-level task into a **directed acyclic graph (DAG)** of sub-tasks, assigns each to an appropriate worker, and tracks dependencies. This is the multi-agent analog of classical hierarchical task network (HTN) planning.

```

from dataclasses import dataclass, field
from typing import List, Optional
import asyncio

@dataclass
class Task:
    id: str
    description: str
    assigned_to: str
    dependencies: List[str] = field(default_factory=list)
    status: str = "pending" # pending | running | done | failed
    result: Optional[str] = None

class TaskDAG:
    def __init__(self):
        self.tasks: dict[str, Task] = {}

    def add_task(self, task: Task):
        self.tasks[task.id] = task

    def ready_tasks(self) -> List[Task]:
        """Return tasks whose dependencies are all completed."""

```

```

    return [
        t for t in self.tasks.values()
        if t.status == "pending"
        and all(self.tasks[d].status == "done"
                for d in t.dependencies)
    ]

    async def execute(self, agent_pool: dict):
        while any(t.status != "done" for t in self.tasks.values()):
            ready = self.ready_tasks()
            if not ready:
                await asyncio.sleep(0.1)
                continue
            # Execute ready tasks in parallel
            await asyncio.gather(*[
                self._run_task(t, agent_pool[t.assigned_to])
                for t in ready
            ])

    async def _run_task(self, task: Task, agent):
        task.status = "running"
        try:
            task.result = await agent.execute(task.description)
            task.status = "done"
        except Exception as e:
            task.status = "failed"
            raise

```

Listing 25.2: Task DAG decomposition

25.3.4 Voting and Consensus

When multiple agents produce conflicting outputs, voting mechanisms aggregate their responses into a single decision. Common schemes include:

- **Majority voting:** the most common answer wins; effective for factual questions
- **Weighted voting:** agents with higher track records or confidence scores receive more weight
- **Debate-based resolution:** agents argue for their positions; a judge agent decides
- **Delphi method:** iterative rounds where agents revise their answers after seeing others' reasoning

Formally, given n agents producing outputs $\{o_1, \dots, o_n\}$ with weights $\{w_1, \dots, w_n\}$, the weighted consensus is:

$$o^* = \arg \max_o \sum_{i=1}^n w_i \cdot \mathbf{1}[o_i = o] \quad (25.1)$$

For continuous outputs (e.g., probability estimates), weighted averaging applies:

$$\hat{p} = \frac{\sum_{i=1}^n w_i \cdot p_i}{\sum_{i=1}^n w_i} \quad (25.2)$$

25.3.5 Market-Based Coordination

Market mechanisms allocate tasks and resources through **auctions and bidding**. The Contract Net Protocol [334], one of the oldest multi-agent coordination mechanisms, is a task auction:

1. A *manager* broadcasts a task announcement with requirements
2. *Contractor* agents submit bids (capability declarations + cost estimates)

3. The manager awards the contract to the best bidder
4. The winning contractor executes and reports results

In LLM systems, bids can be expressed in natural language (“I can complete this in 3 steps with high confidence”) or structured formats. Market mechanisms are particularly effective for resource-constrained settings where API costs must be minimized.

25.3.6 Stigmergy: Indirect Communication Through Environment

Stigmergy [117] replaces explicit agent-to-agent messaging with a simpler mechanism: each agent modifies the shared environment as a side effect of its work, and other agents react to those modifications rather than to direct signals. The classic illustration is a foraging ant depositing pheromone on its return path; subsequent ants amplify successful routes without any ant “talking” to another.

In LLM multi-agent systems, stigmergy manifests as:

- **Shared documents:** agents write to a shared document; others read and build upon it
- **Code repositories:** one agent commits code; another reads and extends it
- **Annotation layers:** agents annotate shared artifacts (highlight errors, add comments)
- **Task queues:** agents add and consume tasks from a shared queue

Stigmergy enables coordination without explicit communication overhead—agents simply observe the state of the shared environment and act accordingly.

25.4 Communication Protocols

Effective multi-agent systems require well-defined communication protocols: agreed-upon formats, semantics, and patterns for agent-to-agent messages. (For the standardized inter-agent protocol, see Chapter 24.)

25.4.1 Structured Message Formats

Messages between LLM agents should be structured to enable reliable parsing and routing. A minimal message schema:

```
from pydantic import BaseModel, Field
from typing import Literal, Optional, Dict, Any
from datetime import datetime, timezone
import uuid

PerformativeType = Literal[
    "inform",      # Share information
    "request",     # Request an action
    "propose",     # Propose a course of action
    "accept",      # Accept a proposal
    "reject",      # Reject a proposal
    "query",       # Ask a question
    "confirm",     # Confirm receipt/completion
    "failure",     # Report a failure
]

class AgentMessage(BaseModel):
    message_id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    conversation_id: str          # Groups related messages
    sender: str                  # Agent identifier
    receiver: str                # Target agent (or "broadcast")
```

```

performative: PerformativeType
content: str # Natural language content
metadata: Dict[str, Any] = {} # Structured payload
reply_to: Optional[str] = None # message_id being replied to
timestamp: datetime = Field(default_factory=lambda: datetime.now(timezone.utc))

def to_llm_prompt(self) -> str:
    """Render message as a prompt fragment for the receiving agent."""
    return (
        f"[MESSAGE from {self.sender}]\n"
        f"Type: {self.performative}\n"
        f"Content: {self.content}\n"
        + (f"Metadata: {self.metadata}\n" if self.metadata else "")
    )

```

Listing 25.3: Agent message schema

25.4.2 Performative Types (FIPA-ACL Inspired)

Drawing from the FIPA Agent Communication Language [93], modernized for LLM agents:

Table 25.1: FIPA-ACL-inspired performative types for LLM agent messages.

Performative	Semantics	Example Use
inform	Sender believes ϕ is true	Share research findings
request	Sender wants receiver to do α	Delegate a sub-task
propose	Sender proposes plan π	Suggest an approach
accept	Receiver agrees to proposal	Confirm task assignment
reject	Receiver declines proposal	Refuse incompatible task
query	Sender wants to know ϕ	Ask for clarification
confirm	Sender confirms ϕ occurred	Acknowledge completion
failure	Sender failed to achieve α	Report error

25.4.3 Context Sharing Strategies

A critical challenge in multi-agent communication is **context management**: how much history does each agent need? Three strategies:

- **Full history**: pass the entire conversation history to each agent. Maximally informative but expensive; context windows fill quickly.
- **Summary**: a summarizer agent condenses prior exchanges into a compact summary. Efficient but lossy; important details may be dropped.
- **Relevant excerpt**: retrieve only the most relevant prior messages using semantic search. Balances cost and informativeness; requires a retrieval mechanism.

Context Sharing Rule of Thumb

Use **full history** for short conversations (<10 turns); **summaries** for medium-length conversations; **retrieval-augmented excerpts** for long-running agent sessions. Always include the most recent k messages verbatim to preserve immediate context.

25.5 Role Design and Specialization

The design of agent roles—their capabilities, personas, and responsibilities—is as much an art as a science. Well-designed roles enable specialization; poorly designed roles create confusion and redundancy.

25.5.1 Defining Agent Roles

Common roles in LLM multi-agent systems:

Table 25.2: Common agent roles in LLM multi-agent systems.

Role	Primary Capability	Typical Tools
Researcher	Information gathering, synthesis	Web search, RAG, databases
Planner	Task decomposition, scheduling	None (reasoning only)
Coder	Code generation, debugging	Code interpreter, linter
Reviewer	Quality assessment, critique	None (reasoning only)
Tester	Test generation, execution	Test runner, coverage tools
Writer	Prose generation, editing	Grammar checker, style guide
Critic	Adversarial evaluation	None (reasoning only)
Orchestrator	Coordination, delegation	All agent interfaces

25.5.2 Capability-Based vs. Role-Based Assignment

Two philosophies for task assignment:

- **Role-based:** tasks are assigned based on predefined role labels. Simple and predictable; may be suboptimal when a task spans multiple roles.
- **Capability-based:** tasks are assigned based on a dynamic assessment of each agent’s capabilities relative to the task requirements. More flexible; requires a capability registry and matching mechanism.

25.5.3 Dynamic Role Reassignment

In long-running systems, static role assignments become suboptimal. Dynamic reassignment allows agents to take on new roles based on:

- Current workload (load balancing)
- Demonstrated performance on recent tasks
- Changing task requirements
- Agent failures requiring coverage

25.5.4 Persona Design for Diversity of Thought

A subtle but powerful technique: give agents **distinct personas** that encourage diverse perspectives. Rather than five identical “assistant” agents, design:

- An *optimist* who emphasizes opportunities
- A *skeptic* who challenges assumptions
- A *pragmatist* who focuses on implementation
- A *visionary* who thinks long-term
- A *devil’s advocate* who argues the opposite position

This diversity of thought, inspired by techniques like Six Thinking Hats [28], reduces groupthink and produces more robust collective reasoning.

Role Conflict Resolution

When agents have overlapping responsibilities, conflicts arise. Resolve them with explicit **priority rules**: define which role takes precedence for each task type. Alternatively, use a **meta-agent** whose sole responsibility is conflict arbitration. Never leave role conflicts implicit—they will manifest as contradictory outputs or infinite loops.

25.6 Multi-Agent Patterns for LLMs

Beyond architectural topologies, several **interaction patterns** have proven particularly effective for LLM-based multi-agent systems. (These complement the single-agent design patterns in Chapter 20.)

25.6.1 Debate Pattern

Multiple agents argue for different positions; a judge agent evaluates the arguments and decides. Debate has been shown to improve factual accuracy and reduce hallucinations [78].

```
async def debate_round(question: str, agents: list, judge: Agent,
                      rounds: int = 2) -> str:
    """Run a multi-agent debate and return the judge's verdict."""
    positions = {a.name: await a.generate_position(question)
                 for a in agents}

    for round_num in range(rounds):
        # Each agent sees others' positions and can rebut
        rebuttals = {}
        for agent in agents:
            others = {k: v for k, v in positions.items()
                      if k != agent.name}
            rebuttals[agent.name] = await agent.rebut(
                question, positions[agent.name], others
            )
        positions = rebuttals

    # Judge evaluates all final positions
    verdict = await judge.evaluate(question, positions)
    return verdict
```

Listing 25.4: Debate pattern implementation

25.6.2 Reflection Pattern

One agent generates an output; a second agent critiques it; the first agent revises based on the critique. This implements a generate-critique-revise loop that iteratively improves quality.

```
async def reflection_loop(task: str, generator: Agent,
                         critic: Agent, max_rounds: int = 3) -> str:
    draft = await generator.generate(task)

    for _ in range(max_rounds):
        critique = await critic.critique(task, draft)
        if critique.is_satisfactory:
            break
        draft = await generator.revise(task, draft, critique.feedback)

    return draft
```

Listing 25.5: Reflection pattern

25.6.3 Division of Labor Pattern

The task is decomposed into independent sub-tasks executed in parallel. Results are aggregated by a synthesis agent. This pattern maximizes throughput for embarrassingly parallel tasks.

25.6.4 Pipeline Pattern

Agents form a sequential processing chain: each agent transforms the output of the previous agent. Analogous to Unix pipes. Effective for tasks with clear sequential dependencies (e.g., research → outline → draft → edit → format).

25.6.5 Ensemble Pattern

Multiple agents independently solve the same problem; a selection mechanism picks the best answer (best-of- N) or aggregates answers (mixture-of-experts style). Improves reliability at the cost of compute.

$$o^* = \arg \max_{o \in \{o_1, \dots, o_N\}} \text{score}(o, \text{task}) \quad (25.3)$$

where score can be a reward model, a judge LLM, or a verifier.

25.6.6 Teacher-Student Pattern

A more capable agent (teacher) guides a less capable agent (student) through a task, providing hints, corrections, and explanations. This pattern enables knowledge distillation at inference time and can be used to fine-tune the student agent.

25.6.7 Red Team Pattern

An adversarial agent (red team) actively tries to find weaknesses, errors, or safety violations in the outputs of other agents. The red team agent is prompted to be maximally critical and creative in its attacks. This pattern is essential for safety-critical applications.

Red Team Agent Prompt

```
RED_TEAM_PROMPT = """You are a red team agent. Your job is to find
flaws, errors, biases, safety violations, and failure modes in the
following output. Be adversarial, creative, and thorough.

Consider:
1. Factual errors or hallucinations
2. Logical inconsistencies
3. Safety and ethical concerns
4. Edge cases the solution doesn't handle
5. Ways a malicious user could exploit this output
6. Unintended consequences

Output: {agent_output}

Provide a detailed critique with specific examples of each flaw found."""
```

25.7 Training Multi-Agent Systems with Reinforcement Learning

Training multi-agent systems with RL introduces challenges that go beyond single-agent RL. The fundamental difficulty is that each agent's environment includes other learning agents, making the environment **non-stationary** from any single agent's perspective.

25.7.1 Mathematical Formulation

A multi-agent system is formalized as a **Markov Game** (also called a stochastic game) [324]:

$$\mathcal{G} = \langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i \in \mathcal{N}}, \mathcal{T}, \{R^i\}_{i \in \mathcal{N}}, \gamma \rangle \quad (25.4)$$

where $\mathcal{N} = \{1, \dots, n\}$ is the set of agents, $\mathcal{S}$ is the shared state space, $\mathcal{A}^i$ is agent i 's action space, $\mathcal{T} : \mathcal{S} \times \mathcal{A}^1 \times \dots \times \mathcal{A}^n \rightarrow \Delta(\mathcal{S})$ is the transition function, $R^i : \mathcal{S} \times \mathcal{A}^1 \times \dots \times \mathcal{A}^n \rightarrow \mathbb{R}$ is agent i 's reward function, and γ is the discount factor.

Each agent i seeks to maximize its expected discounted return:

$$J^i(\pi^1, \dots, \pi^n) = \mathbb{E}_{\pi^1, \dots, \pi^n} \left[\sum_{t=0}^{\infty} \gamma^t R^i(s_t, a_t^1, \dots, a_t^n) \right] \quad (25.5)$$

25.7.2 Independent Learning

The simplest approach: each agent i treats other agents as part of its environment and optimizes its own policy π^i independently using standard single-agent RL (e.g., PPO, REINFORCE).

$$\nabla_{\theta^i} J^i \approx \mathbb{E} \left[\nabla_{\theta^i} \log \pi^i(a_t^i | o_t^i) \cdot \hat{A}_t^i \right] \quad (25.6)$$

Non-Stationarity Problem

Independent learning violates the Markov assumption: as other agents update their policies, the transition and reward distributions seen by agent i change. This can cause training instability, oscillation, and failure to converge. Independent learning works in practice for simple cooperative tasks but struggles in competitive or complex cooperative settings.

25.7.3 Centralized Training, Decentralized Execution (CTDE)

CTDE [239, 306] is the dominant paradigm for cooperative multi-agent RL. During training, a centralized critic has access to the global state s and all agents' actions $\mathbf{a} = (a^1, \dots, a^n)$. During execution, each agent acts using only its local observation o^i .

The centralized critic for agent i :

$$Q_{\phi}^i(s, \mathbf{a}) = Q_{\phi}^i(s, a^1, \dots, a^n) \quad (25.7)$$

The decentralized actor for agent i :

$$\pi_{\theta^i}^i(a^i | o^i) \quad (25.8)$$

The policy gradient with centralized critic:

$$\nabla_{\theta^i} J^i = \mathbb{E} \left[\nabla_{\theta^i} \log \pi^i(a^i | o^i) \cdot Q_{\phi}^i(s, \mathbf{a}) \right] \quad (25.9)$$

CTDE resolves non-stationarity during training (the centralized critic sees the full joint state) while preserving decentralized execution (no communication required at inference time).

25.7.4 Communication Learning

Rather than using fixed communication protocols, agents can **learn what to communicate**. In differentiable communication frameworks [66, 343], agents produce continuous communication vectors m_t^i that are passed to other agents:

$$a_t^i, m_t^i = \pi_{\theta^i}^i(o_t^i, \{m_{t-1}^j\}_{j \neq i}) \quad (25.10)$$

The communication vectors are optimized end-to-end via backpropagation through the joint reward signal. For LLM agents, this is approximated by training agents to produce structured natural language messages that maximize task performance.

25.7.5 Emergent Communication

When agents are trained from scratch with only a reward signal (no predefined language), they can develop **emergent communication protocols** [199]: shared symbol systems that encode task-relevant information. While fascinating scientifically, emergent communication in LLM systems is typically undesirable—we want agents to communicate in human-interpretable language.

25.7.6 Self-Play

In competitive or mixed-motive settings, **self-play** [332] trains agents by having them compete against copies of themselves. This generates an automatic curriculum: as the agent improves, its opponent (a previous version of itself) becomes harder to beat.

For LLM agents, self-play is used in:

- Red team vs. blue team training
- Debate training (agents argue against each other)
- Negotiation training (agents negotiate with each other)

25.7.7 Population-Based Training

Population-Based Training (PBT) [158] maintains a diverse population of agents with different policies, hyperparameters, and specializations. Agents are periodically evaluated; underperforming agents are replaced by mutated copies of high-performing agents.

For multi-agent LLM systems, PBT enables:

- Automatic discovery of effective role specializations
- Robustness to individual agent failures (diverse population)
- Avoidance of local optima through population diversity

25.7.8 Social Welfare and Nash Equilibrium

In multi-agent settings, the notion of optimality is more complex than in single-agent settings. Two key solution concepts:

Nash Equilibrium: a joint policy $(\pi^{1*}, \dots, \pi^{n*})$ such that no agent can improve its expected return by unilaterally deviating:

$$J^i(\pi^{i*}, \pi^{-i*}) \geq J^i(\pi^i, \pi^{-i*}) \quad \forall i, \forall \pi^i \quad (25.11)$$

where π^{-i} denotes the joint policy of all agents except i .

Social Welfare Maximization: optimize the sum of all agents’ returns:

$$\max_{\pi^1, \dots, \pi^n} \sum_{i=1}^n J^i(\pi^1, \dots, \pi^n) \quad (25.12)$$

In fully cooperative settings (all agents share the same reward), social welfare maximization is the appropriate objective. In competitive settings, Nash equilibrium is the relevant solution concept. Most real-world multi-agent LLM systems are **mixed-motive**: agents have partially aligned, partially conflicting objectives.

Further Reading: Game Theory for Multi-Agent RL

For readers interested in the game-theoretic foundations of multi-agent systems:

- **Shoham & Leyton-Brown** [328] — comprehensive textbook covering Nash equilibria, mechanism design, and social choice theory for agent systems.

- **Zhang et al. [423]** — survey of multi-agent RL algorithms with convergence guarantees under cooperative, competitive, and mixed settings.
- **Nisan et al. [267]** — the definitive reference on algorithmic game theory, covering auctions, equilibria computation, and price of anarchy.

25.8 Challenges and Solutions

25.8.1 Coordination Overhead

Every inter-agent message consumes tokens—and therefore time and money. In a naive implementation, agents communicate constantly, even when unnecessary.

When NOT to Communicate

- When the information is already in the shared blackboard
- When the receiving agent doesn’t need the information for its current task
- When the message would duplicate information already sent
- When the task is simple enough for a single agent

Rule: communicate only when the expected value of the information exceeds the cost of the message.

Quantifying communication cost: if a message costs c tokens and the receiving agent’s task has value v , communicate only if the expected improvement in task value $\Delta v > c \cdot \text{cost_per_token}$.

25.8.2 Redundancy vs. Efficiency

Multiple agents may independently solve the same sub-problem, wasting compute. Solutions:

- **Duplicate detection:** before starting a task, check the blackboard for existing results
- **Result caching:** store completed sub-task results with semantic keys for retrieval
- **Task locking:** mark tasks as “in progress” to prevent duplicate execution

25.8.3 Attribution

When a multi-agent system succeeds or fails, which agent is responsible? Attribution is critical for:

- RL reward assignment (credit assignment problem)
- Debugging and improvement
- Trust calibration (which agents to rely on)

The **counterfactual credit assignment** approach estimates each agent’s contribution by asking: “How much would the outcome have changed if this agent had acted differently?”

$$\text{credit}^i = J(\pi^1, \dots, \pi^n) - J(\pi^1, \dots, \pi_{\text{default}}^i, \dots, \pi^n) \quad (25.13)$$

25.8.4 Scalability

Naive message passing scales as $O(n^2)$ with the number of agents. Solutions:

- **Hierarchical communication:** agents communicate only within their subtree
- **Topic-based pub/sub:** agents subscribe only to relevant message topics

- **Sparse communication graphs:** only connect agents that need to interact
- **Asynchronous communication:** agents don’t block waiting for responses

25.8.5 Emergent Behavior and Safety

Multi-agent systems can exhibit unexpected emergent behaviors—interactions between agents that produce outcomes no individual agent was designed to produce. This is both a feature (emergent capabilities) and a risk (emergent failures).

Safety Concerns in Multi-Agent Systems

- **Prompt injection cascades:** a malicious input to one agent propagates through the system
- **Reward hacking:** agents find unexpected ways to maximize reward that violate intent
- **Collusion:** in competitive settings, agents may develop implicit collusion strategies
- **Amplification:** errors or biases in one agent are amplified by downstream agents

Always include a **safety monitor agent** that observes all inter-agent communications and can halt the system if unsafe behavior is detected.

25.8.6 Evaluation

Evaluating multi-agent systems requires metrics at multiple levels:

Table 25.3: Multi-level evaluation metrics for multi-agent systems.

Level	Metric	Example
System	Task completion rate	% of tasks completed correctly
System	End-to-end latency	Time from task to final output
System	Total token cost	Tokens consumed across all agents
Agent	Individual accuracy	Per-agent task success rate
Agent	Communication efficiency	Useful messages / total messages
Agent	Contribution score	Counterfactual credit (Eq. ??)
Emergent	Coordination quality	Degree of task overlap / gaps

25.9 Real-World Multi-Agent Applications

25.9.1 Software Development Team

A multi-agent software development team mirrors a real engineering organization:

```
from dataclasses import dataclass
from typing import Optional
import asyncio

@dataclass
class SoftwareTeamState:
    requirements: str
    architecture: Optional[str] = None
    code: Optional[str] = None
    tests: Optional[str] = None
    review_feedback: Optional[str] = None
    final_code: Optional[str] = None
    approved: bool = False

class SoftwareDevelopmentTeam:
    """
    Multi-agent software team:
```

```

    Architect -> Coder -> Tester -> Reviewer -> (iterate or ship)
    """

    def __init__(self, llm_factory):
        self.architect = llm_factory(
            system_prompt="""You are a software architect. Given requirements,
            produce a clear technical design: components, interfaces, data
            structures, and implementation plan."""
        )
        self.coder = llm_factory(
            system_prompt="""You are an expert software engineer. Given a
            technical design, write clean, well-documented, production-ready
            code. Follow best practices for the language."""
        )
        self.testers = llm_factory(
            system_prompt="""You are a QA engineer. Given code, write
            comprehensive tests: unit tests, edge cases, integration tests.
            Identify potential bugs and failure modes."""
        )
        self.reviewer = llm_factory(
            system_prompt="""You are a senior code reviewer. Evaluate code
            for correctness, security, performance, and maintainability.
            Provide specific, actionable feedback. Approve only if excellent."""
        )

    async def build(self, requirements: str,
                    max_iterations: int = 3) -> SoftwareTeamState:
        state = SoftwareTeamState(requirements=requirements)

        # Phase 1: Architecture
        state.architecture = await self.architect.invoke(
            f"Requirements:\n{requirements}\n\nProduce technical design."
        )

        for iteration in range(max_iterations):
            # Phase 2: Implementation
            prompt = (f"Design:\n{state.architecture}\n\n"
                     + (f"Previous feedback:\n{state.review_feedback}\n\n"
                        if state.review_feedback else ""))
            prompt += "Write the implementation."
            state.code = await self.coder.invoke(prompt)

            # Phase 3: Testing
            state.tests = await self.testers.invoke(
                f"Code:\n{state.code}\n\nWrite comprehensive tests."
            )

            # Phase 4: Review
            review = await self.reviewer.invoke(
                f"Code:\n{state.code}\n\nTests:\n{state.tests}\n\n"
                "Review this code. End with APPROVED or NEEDS_REVISION."
            )

            if "APPROVED" in review:
                state.final_code = state.code
                state.approved = True
                break
            else:
                state.review_feedback = review

        return state

    async def run(self, requirements: str) -> str:
        state = await self.build(requirements)
        if state.approved:
            return f"# Final Implementation\n\n{state.final_code}"

```

```
else:
    return f"# Best Attempt (not approved)\n\n{state.code}"
```

25.9.2 Research Team

A research team agent society mirrors academic collaboration:

- **Literature Reviewer:** searches and synthesizes existing work
- **Hypothesis Generator:** proposes novel research directions
- **Experimentalist:** designs and runs experiments (via code execution)
- **Statistician:** analyzes results and assesses significance
- **Writer:** synthesizes findings into a coherent report

25.9.3 Customer Service System

A tiered customer service system:

- **Router:** classifies incoming requests and routes to specialists
- **Billing Specialist:** handles payment and account issues
- **Technical Specialist:** resolves product/service issues
- **Escalation Agent:** handles complex cases requiring human judgment

25.9.4 Creative Team

A creative production pipeline:

- **Brainstormer:** generates diverse ideas without self-censorship
- **Drafter:** develops the most promising ideas into full drafts
- **Editor:** refines drafts for clarity, style, and coherence
- **Critic:** provides adversarial feedback to strengthen the work

25.10 Architecture Comparison

Table 25.4: Multi-agent architecture patterns compared across key dimensions. Ratings: **H**igh / **M**edium / **L**ow.

Architecture	Scalability	Debug	Coord.	Cost	Fault Tol.	Best For
Centralized (Supervisor)	M	H	L		L	Simple pipelines; clear task decomposition; small teams
Decentralized (P2P)	H	L	H		H	Dynamic environments; resilience-critical; large-scale
Hierarchical	H	M	M		M	Enterprise workflows; complex multi-domain tasks
Swarm	H	L	L		H	Customer service routing; simple handoff chains
Pipeline	M	H	L		L	Sequential processing; clear stage dependencies
Ensemble	L	H	H		H	High-stakes decisions; reliability over efficiency

Choosing an Architecture

- **Independent sub-tasks** → parallel architectures (division of labor, ensemble).
- **Sequential with clear dependencies** → pipeline.
- **Fault tolerance required** → avoid centralized; prefer hierarchical or decentralized.

- **Debuggability critical** → centralized or pipeline (all decisions traceable).
- **<5 agents** → centralized is simplest. **>20 agents** → hierarchical or swarm.

In practice, most production systems use **hierarchical** architectures: a top-level supervisor delegates to domain-specific sub-supervisors, who manage small teams of specialized workers.

25.11 Self-Evolving Agents: BDI Meets LLMs

A May 2026 line of research [427] represents the first serious attempt to combine **Belief-Desire-Intention (BDI)** architectures—the classical framework for rational agents in multi-agent systems—with LLMs. Unlike static agents that execute fixed tool-call sequences, BDI-LLM agents autonomously modify their own goals and rewrite their underlying code based on environmental feedback:

- **Beliefs:** The LLM maintains and updates a structured world model (beliefs about file states, API responses, user intent)
- **Desires:** High-level goals that can be dynamically reprioritized based on observed outcomes
- **Intentions:** Concrete plans (code + tool sequences) that the agent can rewrite when a plan fails

The Alignment Challenge of Self-Modification

Early experiments revealed a critical failure mode: agents deleting their own safety constraints to speed up execution. When an agent can rewrite its own code, alignment becomes a moving target—the guardrails you installed may not survive the agent's self-optimization. This places BDI-LLM architectures at the frontier of agent autonomy but demands robust meta-level constraints that the agent cannot modify.

25.12 Summary

Multi-agent systems represent a fundamental shift in how we deploy LLMs: from isolated assistants to collaborative societies of specialized agents. The key insights from this section:

Multi-Agent Systems: Key Takeaways

1. **Architecture matters:** the topology of agent connections determines scalability, debuggability, and fault tolerance. Choose based on task structure and operational requirements.
2. **Coordination is expensive:** every inter-agent message costs tokens. Design communication protocols to minimize overhead while preserving necessary information flow.
3. **Specialization enables quality:** agents with focused roles and tailored prompts consistently outperform generalist agents on complex tasks.
4. **RL training is hard:** multi-agent RL introduces non-stationarity, credit assignment challenges, and emergent behaviors. CTDE is the current best practice for cooperative settings.
5. **Safety requires explicit design:** multi-agent systems can amplify errors and exhibit unexpected emergent behaviors. Safety monitoring must be a first-class architectural concern.
6. **Start simple:** begin with a centralized supervisor pattern, measure its limitations, and evolve toward more complex architectures only when necessary.

The field of multi-agent LLM systems is evolving rapidly. The patterns and techniques described here represent the current state of the art, but new architectures, coordination mechanisms, and